Python: `for` Loops

Cheng-Te Li (李政德)

chengte@mail.ncku.edu.tw

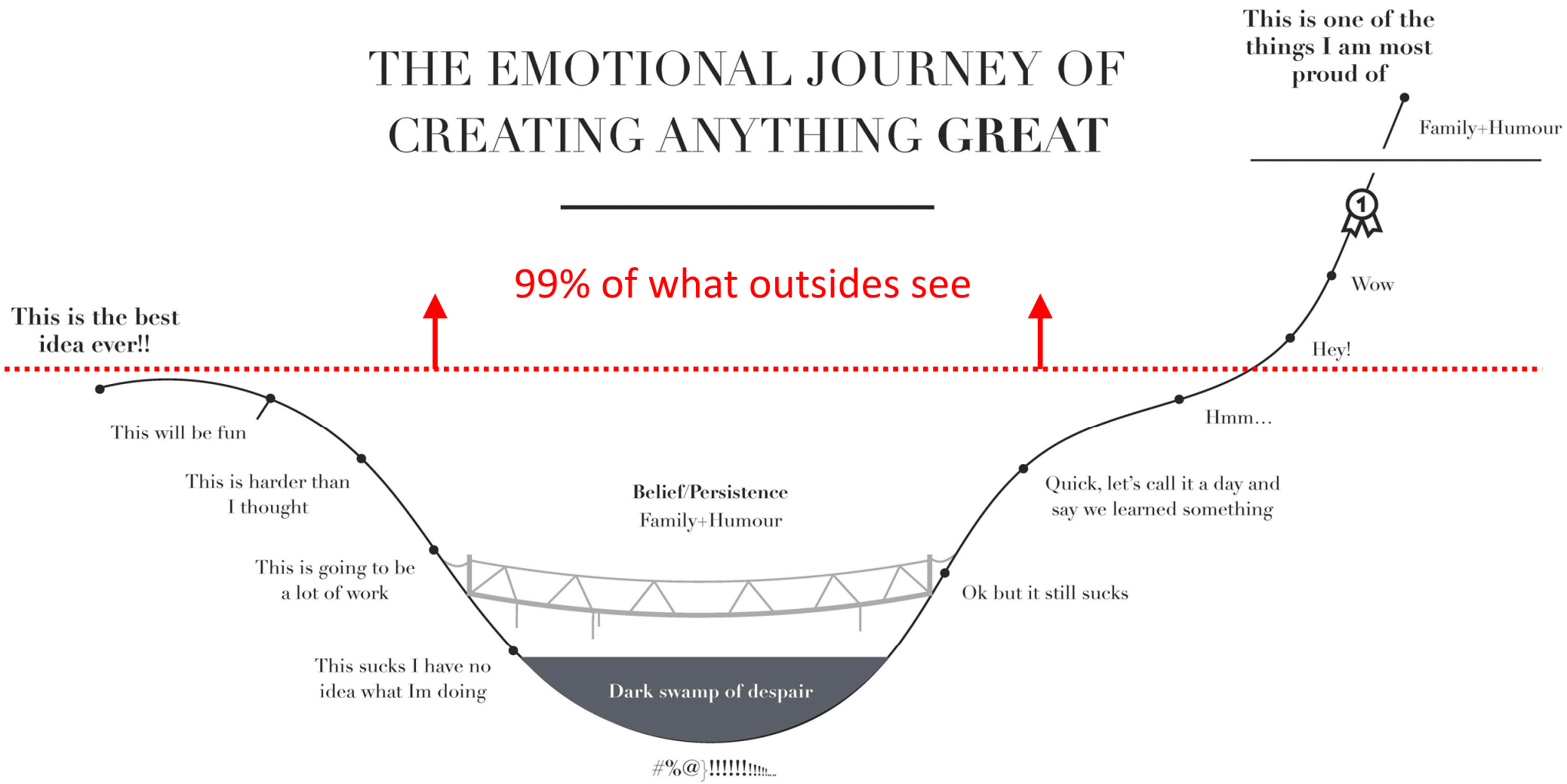
Dept. of Statistics, NCKU

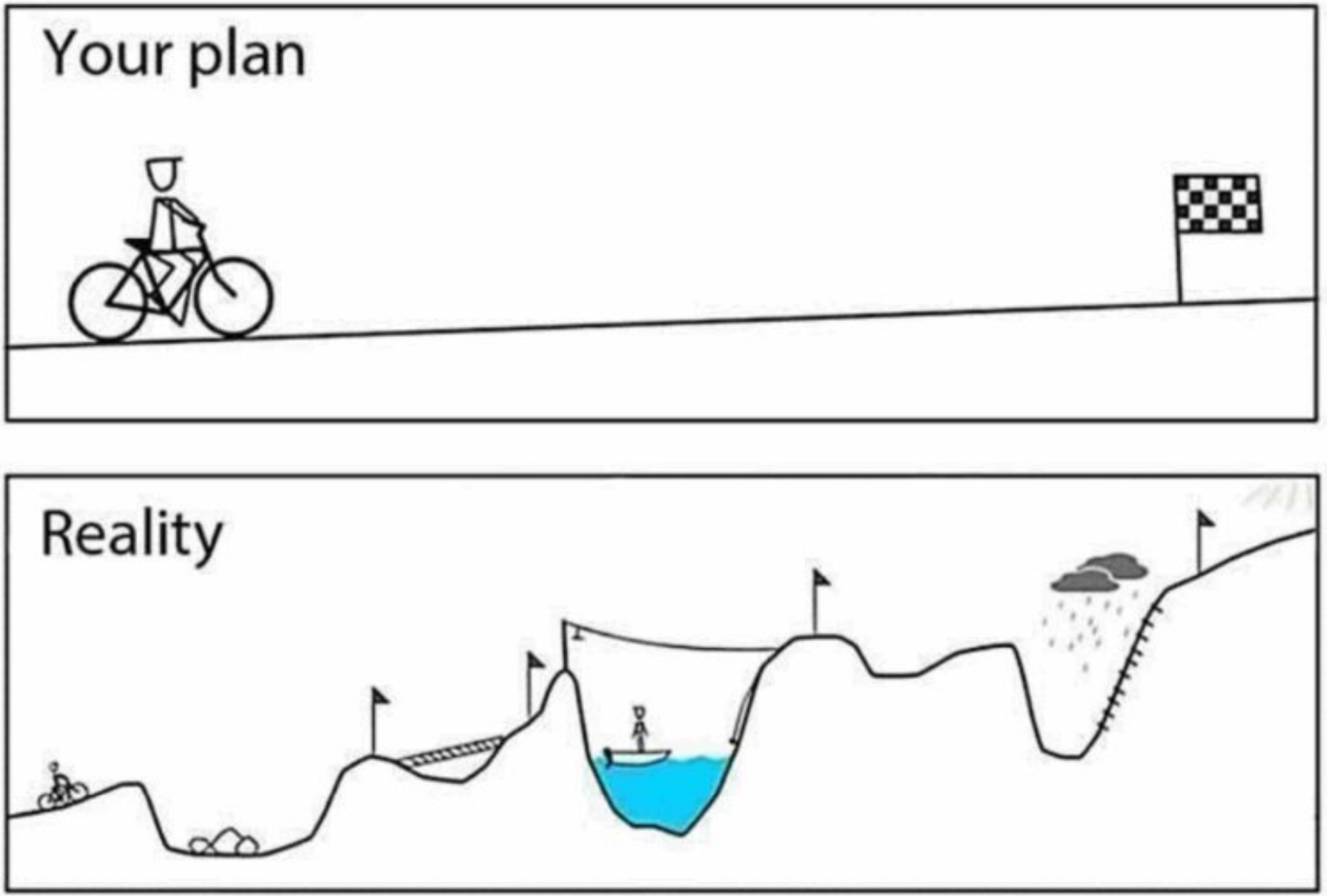
Course Info (12/3)

- No worries for HW3 and exams

  -- Just try your best! ☺
- Course schedule
  - HW4: 12/3 – 12/23 (for loop, basic functions)
  - Quiz3: 12/31
  - Final Exam: 1/7
- The following weeks
  - Lecture 6: for loop
  - Lecture 7: functions (basic)
  - Lecture 8: files
  - Lecture 9: dictionary

「你必須很努力，才能看起來毫不費力」




Your plan
Reality

Math Functions

- `abs()`: return the absolute value
- `max()`: return the maximum value (in a list)
- `min()`: return the minimum value (in a list)
- `round()`: return the round-off value

```
x, y, z = -17, 5, 100
print(abs(x))                       # 17
print(pow(y, 2))                    # 25
print(pow(z, 0.5))                  # 10.0
print(max(8, 5, 1, 7, 3))           # 8
print(max([8, 5, 1, 7, 3, 12]))     # 12
print(min(5, 6, 1, 4, 2))           # 1
print(min([6, -9, 2, -2]))          # -9
print(round(12.34))                 # 12
print(round(17.89))                 # 18
```

Common Functions

- `abs()`: return the absolute value
- `int()`: change the type to integer
- `float()`: change the type to floating number
- `str()`: change the type to string
- `type()`: return the type

```
x = -17
print(abs(x))           # 17
s = "-17"
print(int(s))           # -17
print(float(s))         # -17.0
print(str(x))           # -17
print(type(x))          # <class 'int'>
print(type(str(x)))     # <class 'str'>
```

Which values are False?

- int/float – `0`, `0.0`: False, otherwise: True
- List – `[ ]`: False, otherwise: True
- String – `""` or `''`: False, otherwise: True
- None – False
- Set – `{}`: False

```
i = 3
while i:
    print(i)
    i = i - 1
# 2
# 1
s = ""
if s:
    print(s)
```

```
s = "abc"
while s:
    s = s[:len(s)-1]
    print(s)

# ab
# a
#
```

```
l = [1, 2, 3]
while l:
    del l[0]
    print(l)

# [2, 3]
# [3]
# []
```

Indefinite vs. Definite Loops

Here's an example of a `while` loop that counts from 0 to 10:

```
i = 0
while i <= 10:
    print(i)
    i = i + 1
```

The code has the same output as this `for` loop:

```
for i in range(11):
    print(i)
```

The `while` loop requires us to manage the loop variable **i** by initializing it to 0 before the loop and incrementing it at the bottom of the body

In the `for` loop this is handled automatically

for loops

- A for loop iterates over the values returned by any **iterable** object – that is, any object that can yield a sequence of values
  - **Iterable** object, ex: list, string, range (and tuple, set, dictionary, generator)

```
[1, 2, 3, 4, 5]
['a', 'b', 'c', 'd']
"hello world"
range(2, 5)
```

- A for loop's general form:

  for ⟨item⟩ **in** ⟨iterable⟩**:**

  ⟨action⟩

  indentation

  - ⟨action⟩ will be executed once for each element of ⟨iterable⟩
  - **variable** is set to be the **first item** of ⟨iterable⟩, and body is executed; then, variable is set to be the **second item** of sequence, and body is executed; and so on, for each remaining **item** of the sequence

for ⟨item⟩ in ⟨iterable⟩:

item

⟨action⟩

排隊為了．．．	
吃美食	
上車搶位置	跟偶像見面 ♥
結帳買東西	看電影/表演
$ $ $ 相關	文藝活動
解決內急	進場看開球

http://dailyview.tw/Daily/2014/07/10

Simple `for` Loops: Example

`for` loop

```
1 for i in [5, 4, 3, 2, 1]:
2     print(i)
3 print("Happy New Year!\n0")
```

`while` loop

```
1 n = 5
2 while n > 0:
3     print(n)
4     n = n - 1
5 print("Happy New Year!")
6 print(n)
```

Output:

```
5
4
3
2
1
Happy New Year!
0
```

Simple `for` Loops: Example

`for` loop

```
1 friends = ['Joseph', 'Glenn', 'Sally']
2 for friend in friends:
3     print("Happy New Year:", friend)
4 print("Done!")
```

Output:

```
Happy New Year: Joseph
Happy New Year: Glenn
Happy New Year: Sally
Done!
```

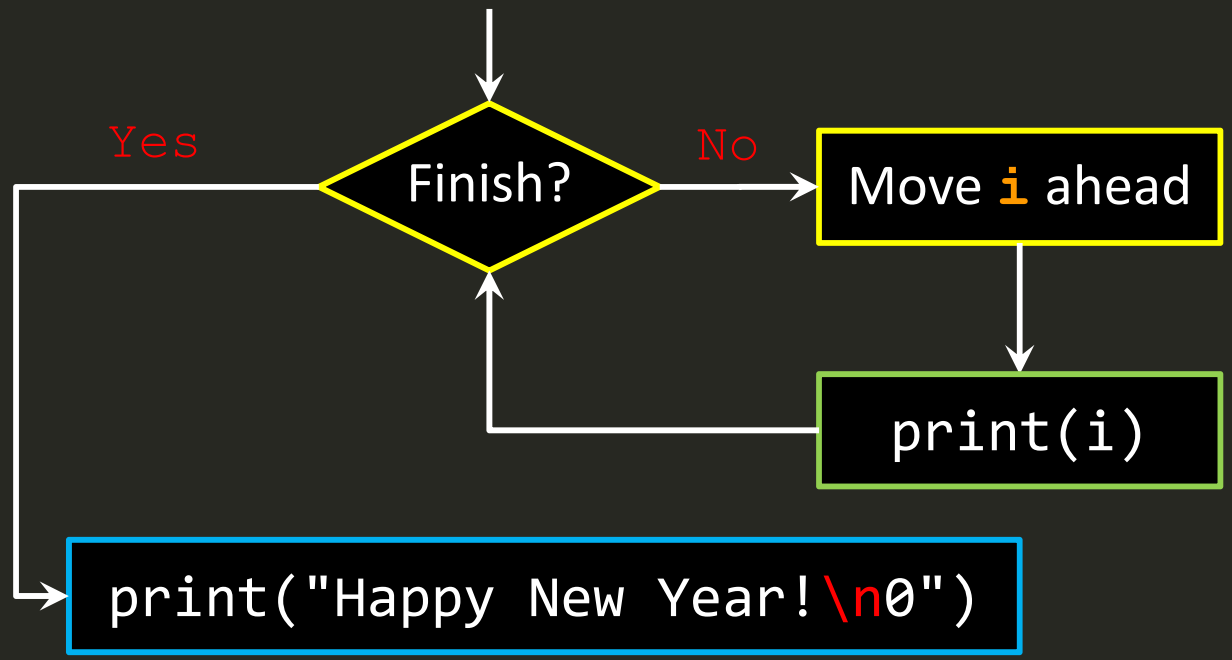
`while` loop

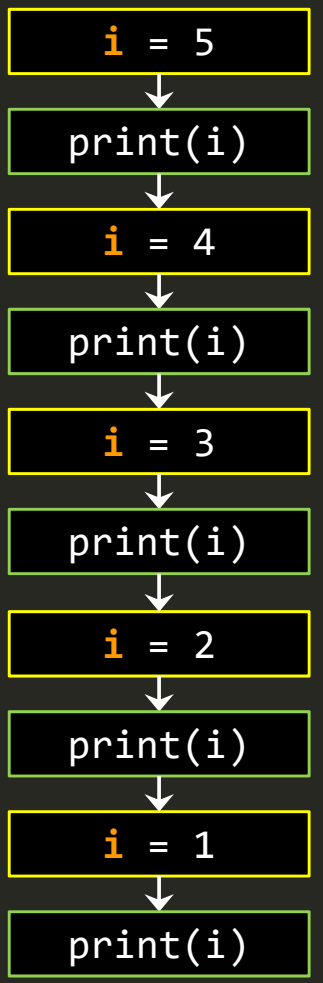
```
1 friends = ['Joseph', 'Glenn', 'Sally']
2 i = 0
3 while i < len(friends):
4     friend = friends[i]
5     print("Happy New Year:", friend)
6     i = i + 1
7 print("Done!")
```

A Simple Definite Loop

- Definite loops (for loops) have explicit iteration variables that change each time through a loop. These iteration variables move through the sequence or set.

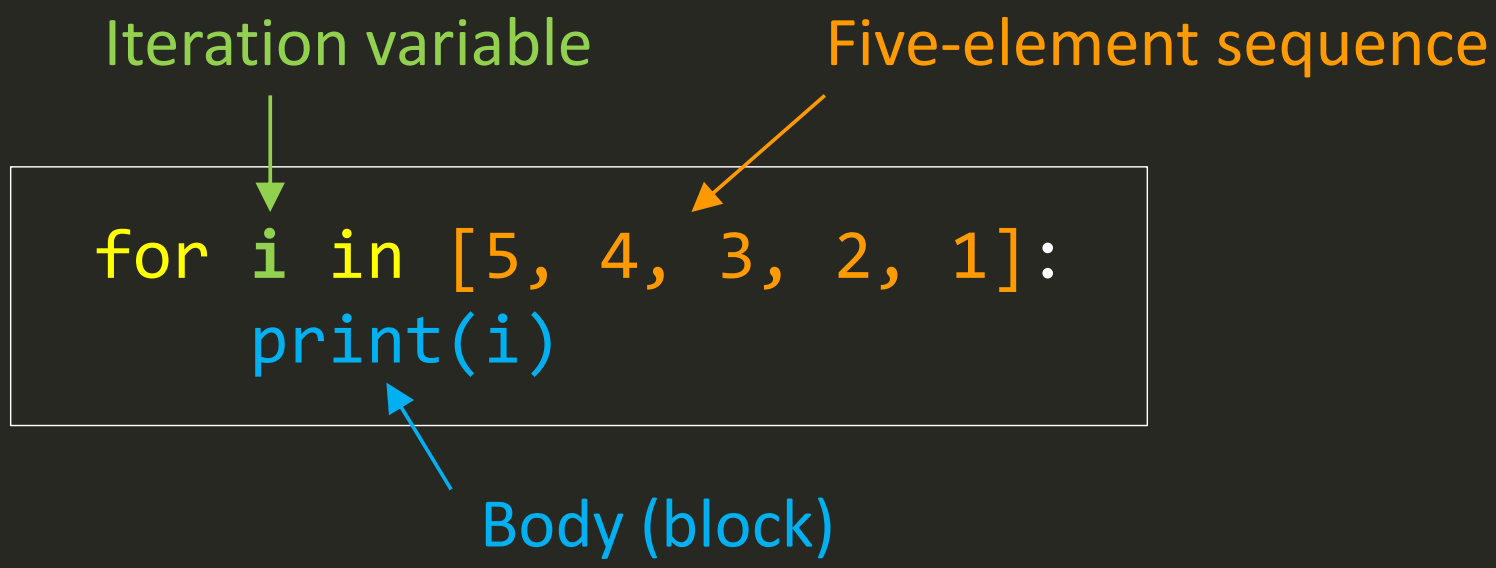
```
1 for i in [5, 4, 3, 2, 1]:
2     print(i)
3 print("Happy New Year!\n0")
```





Look at IN

- The iteration variable “iterates” through the sequence
- The block (body) of code is executed once for each value **in** the sequence
- The iteration variable moves through all of the values **in** the sequence

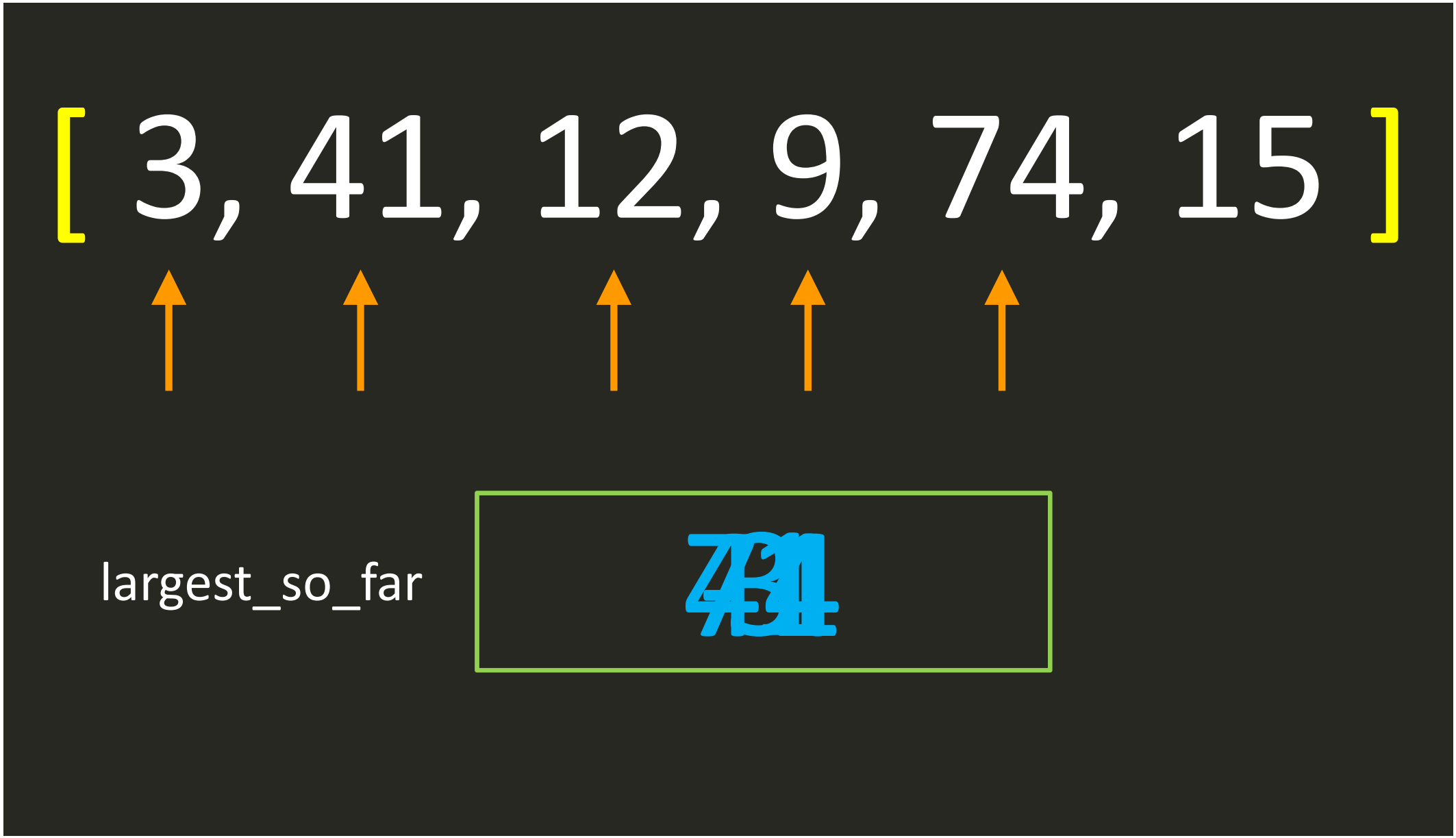


Make **Smart** Loops

- The trick is “knowing” something about the whole loop when you are stuck writing code that only sees one entry at a time
  - Apply to all kinds of loops, `while` and `for`

```
Set some variables to initial values
for thing in data:
    Look for something or do something to each
    entry separately, updating variables
Look at the variables
```

Example: Find the Largest Value in a List



Example: Find the Largest Value in a List

```
1 largest_so_far = -1 # put the largest so far
2 print("Before:", largest_so_far)
3 for i in [9, 41, 12, 3, 74, 15]:
4     # check if i is greater than the largest_so_far
5     # if it is True, update largest_so_far
6     if i > largest_so_far:
7         largest_so_far = i
8     print(largest_so_far, i)
9 print("After:", largest_so_far)
```

```
Before: -1
9 9
41 41
41 12
41 3
74 74
74 15
After: 74
```

We make a variable that contains the largest value we have seen so far
If the current number we are looking at is larger, it is the new largest value we have seen so far

Example: Count in a Loop

```
1 count = 0 # initialization
2 print("Before:", count)
3 for i in [9, 41, 12, 3, 74, 15]:
4     # update the counter
5     count = count + 1
6     print(count, i)
7 print("After:", count)
```

```
Before: 0
1 9
2 41
3 12
4 3
5 74
6 15
After: 6
```

```
1 L = [9, 41, 12, 3, 74, 15]
2 print(len(L))
```

To count how many times we execute a loop, we introduce a counter variable that starts at 0 and we add one to it each time through the loop

Example: Sum up in a Loop

```
1 total = 0 # initialization
2 print("Before:", total)
3 for i in [9, 41, 12, 3, 74, 15]:
4     # add the value to the sum
5     total = total + i
6     print(total, i)
7 print("After:", total)
```

```
Before: 0
9 9
50 41
62 12
65 3
139 74
154 15
After: 154
```

```
1 L = [9, 41, 12, 3, 74, 15]
2 print(sum(L))
```

To add up a value we encounter in a loop, we introduce a sum variable that starts at 0 and we add the value to the sum each time through the loop

Example: Find the Average in a Loop

```
1  count = 0
2  total = 0 # initialization
3  print("Before:", count, total)
4  for value in [9, 41, 12, 3, 74, 15]:
5      # update the counter
6      count = count + 1
7      # add the value to the sum
8      total = total + value
9      print(count, total, value)
10 print("After:", count, total, total/count)
```

```
Before: 0 0
1 9 9
2 50 41
3 62 12
4 65 3
5 139 74
6 154 15
After: 6 154 25.666
```

```
1 L = [9, 41, 12, 3, 74, 15]
2 print(sum(L)/len(L))
```

An average just combines the counting and sum patterns and divides when the loop is done

Example: Filtering in a Loop

```
1 L = []
2 print("Before")
3 for value in [9, 41, 12, 3, 74, 15]:
4     if value > 20:
5         print("Large number", value)
6         L.append(value)
7 print("After")
8 print(L)
```

```
Before
Large number 41
Large number 74
After
[41, 74]
```

```
1 L = [9, 41, 12, 3, 74, 15]
2 L.sort()
3 print(L[len(L)-2:])
```

We use an if statement in the loop to catch / filter the values we are looking for

Example: Search Using a Boolean Variable

```
1 found = False
2 print("Before", found)
3 for value in [9, 41, 12, 3, 74, 15]:
4     if value == 3:
5         found = True
6     print(found, value)
7 print("After", found)
```

```
Before False
False 9
False 41
False 12
True 3
True 74
True 15
After True
```

```
1 L = [9, 41, 12, 3, 74, 15]
2 print(3 in L)
```

If we just want to search and know if a value was found, we use a variable that starts at False and is set to True as soon as we find what we are looking for

In-Class Exercise: Find the Smallest Value

[3, 41, 12, 9, 74, 15]

```
1 smallest_so_far = -1
2 print("Before:", smallest_so_far)
3 for value in [9, 41, 12, 3, 74, 15]:
4     if value < smallest_so_far:
5         smallest_so_far = value
6     print(smallest_so_far, value)
7 print("After:", smallest_so_far)
```

```
Before: -1
-1 9
-1 41
-1 12
-1 3
-1 74
-1 15
After: -1
```

On-site Exercise: Find the Smallest Value

tag is not used yet

```
1 smallest_so_far = None
2 print("Before:", smallest_so_far)
3 for value in [9, 41, 12, 3, 74, 15]:
4     if smallest_so_far is None:
5         smallest_so_far = value
6     if value < smallest_so_far:
7         smallest_so_far = value
8     print(smallest_so_far, value)
9 print("After:", smallest_so_far)
```

```
Before: None
9 9
9 41
9 12
3 3
3 74
3 15
After: 3
```

We still have a variable that is the smallest_so_far.
The first time through the loop smallest_so_far is **None**, so we take the first value to be the smallest_so_far.

Revision in Loops

```
1 values = [2, 3, 5, 7]
2 for v in values:
3     v = 2 * v
4 print(values)
```

What will be printed out? [2, 3, 5, 7]

To change a list, we need to repeat statements with indices like `values[0] = 2 * values[0]`

The Range Function

- Use the `range` function to **generate** a list of **integer** values in a range; it works only for ints
- `range()` can return an **iterable object**
- `list()` is used only to force the items range would generate to appear as a list
  - It's not normally used in actual code

`range(stop)` or `range(start, stop, step)`

一定要有 一定要有 可有可無

```
print(range(0, 5))          # range(0, 5)
type(range(0, 5))           # <class 'range'>
list(range(0, 5))           # [0, 1, 2, 3, 4]
print(list(range(0, 5)))    # [0, 1, 2, 3, 4]
list(range(3))              # [0, 1, 2]
list(range(2, 7))           # [2, 3, 4, 5, 6]
list(range(10, 25, 3))      # [10, 13, 16, 19, 22]
```

Examples of Using range()

```
print(range(0, 5))              # range(0, 5)
type(range(0, 5))               # <class 'range'>
list(range(0, 5))               # [0, 1, 2, 3, 4]
print(list(range(0, 5)))        # [0, 1, 2, 3, 4]
list(range(3))                  # [0, 1, 2]
list(range(2, 7))               # [2, 3, 4, 5, 6]
list(range(-3, 4))              # [-3, -2, -1, 0, 1, 2, 3]
list(range(10, 25, 3))          # [10, 13, 16, 19, 22]
list(range(-3, 4, 2))           # [-3, -1, 1, 3]
list(range(-3, 4, -2))          # []
list(range(2020, 2000, -4))     # [2020, 2016, 2012, 2008, 2004]
```

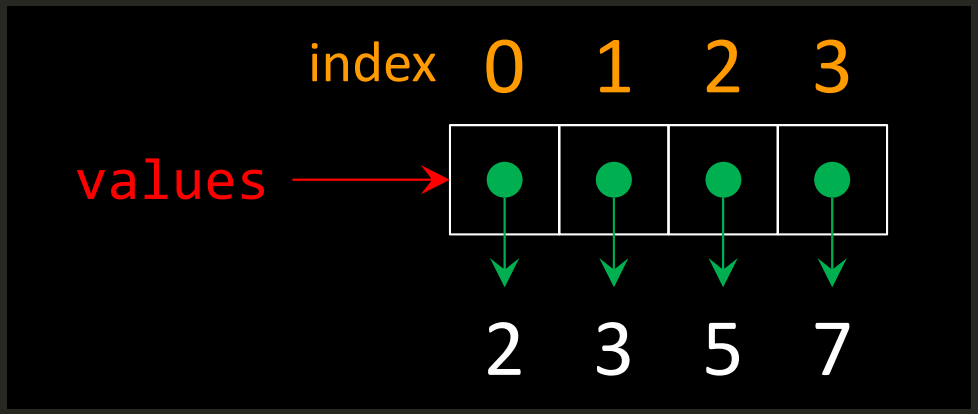
```
1 total = 0
2 for i in range(1,5):
3     print(i)
4     total += i
5 print("sum up range(1,5): ", total)
```

```
1
2
3
4
sum up range(1,5):  10
```

Revision in Loops

```
1 values = [2, 3, 5, 7]
2 for i in range(len(values)):
3     values[i] = 2 * values[i]
4 print(values)
```

What will be printed out? [4, 6, 10, 14]



- With `range()`, we can use for loop to iterate over the indices of a list
- Use `len()` to get the number of elements in a list
- `range(len(list))` gives the list of valid list **indices**
- Bottom line: If you want to change a list element, access it by index

Two Iterating Approaches

Loop variable iterating over the list elements

```
1  nums = [10, 20, 30, 40, 50]
2  total = 0
3  for i in nums:
4      total = total + i
```

Loop variable iterating over the list index values

```
1  nums = [10, 20, 30, 40, 50]
2  total = 0
3  for i in range(len(nums)):
4      total = total + nums[i]
```

Nested `for` Loop

- Nested lists may have unequal lengths
- You can use loop variable to iterate over the list elements

```
1  # the data of drinking time for three days
2  times = [["9:02", "10:17", "13:52", "21:15"],
3                ["8:45", "13:44", "14:13"],
4                ["8:55", "11:11", "12:34", "18:23", "21:31"]]
5  for day in times:
6      for drink_time in day:
7          for ch in drink_time:
8              print(ch)
9          print("---")
10     print()
```

```
9:02 10:17 13:52 21:15
8:45 13:44 14:13
8:55 11:11 12:34 18:23 21:31
```

```
9
:
0
2
---
1
0
:
1
7
---
1
3
:
5
2
---
2
1
:
1
5
---
```

Example: Generate All Sublists of a List

```
1 L = ["a", "b", "c", "d", "e"]
2 i = 0
3 while i < len(L):
4     j = i + 1
5     while j <= len(L):
6         print(L[i:j])
7         j = j + 1
8     i = i + 1
```

while loop

```
['a']
['a', 'b']
['a', 'b', 'c']
['a', 'b', 'c', 'd']
['a', 'b', 'c', 'd', 'e']
['b']
['b', 'c']
['b', 'c', 'd']
['b', 'c', 'd', 'e']
['c']
['c', 'd']
['c', 'd', 'e']
['d']
['d', 'e']
['e']
```

for loop

```
1 L = ["a", "b", "c", "d", "e"]
2 for i in range(len(L)):
3     for j in range(i+1, len(L)+1):
4         print(L[i:j])
```

On-site Exercise: 九九乘法表

```
c:\Python35-32\workspace>python while_99table.py
1 x 1 = 1       2 x 1 = 2       3 x 1 = 3
1 x 2 = 2       2 x 2 = 4       3 x 2 = 6
1 x 3 = 3       2 x 3 = 6       3 x 3 = 9
1 x 4 = 4       2 x 4 = 8       3 x 4 = 12
1 x 5 = 5       2 x 5 = 10      3 x 5 = 15
1 x 6 = 6       2 x 6 = 12      3 x 6 = 18
1 x 7 = 7       2 x 7 = 14      3 x 7 = 21
1 x 8 = 8       2 x 8 = 16      3 x 8 = 24
1 x 9 = 9       2 x 9 = 18      3 x 9 = 27

4 x 1 = 4       5 x 1 = 5       6 x 1 = 6
4 x 2 = 8       5 x 2 = 10      6 x 2 = 12
4 x 3 = 12      5 x 3 = 15      6 x 3 = 18
4 x 4 = 16      5 x 4 = 20      6 x 4 = 24
4 x 5 = 20      5 x 5 = 25      6 x 5 = 30
4 x 6 = 24      5 x 6 = 30      6 x 6 = 36
4 x 7 = 28      5 x 7 = 35      6 x 7 = 42
4 x 8 = 32      5 x 8 = 40      6 x 8 = 48
4 x 9 = 36      5 x 9 = 45      6 x 9 = 54

7 x 1 = 7       8 x 1 = 8       9 x 1 = 9
7 x 2 = 14      8 x 2 = 16      9 x 2 = 18
7 x 3 = 21      8 x 3 = 24      9 x 3 = 27
7 x 4 = 28      8 x 4 = 32      9 x 4 = 36
7 x 5 = 35      8 x 5 = 40      9 x 5 = 45
7 x 6 = 42      8 x 6 = 48      9 x 6 = 54
7 x 7 = 49      8 x 7 = 56      9 x 7 = 63
7 x 8 = 56      8 x 8 = 64      9 x 8 = 72
7 x 9 = 63      8 x 9 = 72      9 x 9 = 81
```

while loop

```
# 9 x 9 table (version 2)
i, j = 1, 1
while i <= 9:
    while j <= 9:
        k = 0
        while k <= 2:
            print(i+k, "x", j, "=", (i+k)*j, end = "\t")
            k = k + 1
        print()
        j = j + 1
    print()
    i = i + 3
    j = 1
```

for loop

```
for i in range(1,10,3):
    for j in range(1,10):
        for k in range(3):
            print(i+k, "x", j, "=", (i+k)*j, end="\t")
        print()
    print()
```

Controlling Loops

- The `break` statement exits the loop body immediately, and execution resumes with the first statement after the loop
  - `break` only exits the innermost loop that contains it
  - In a nested loop, `break` in the inner loop will only exit the inner loop, not both loops
- The `continue` statement immediately starts the next iteration of the loop and skips any statements in the loop body that appear after it

continue

```
1  score = [["Tom", "M", 80], ["Mary", "F", 93],
2           ["Amy", "F", 98], ["John", "M", 95]]
3  avg_F, count_F = 0, 0  # store ths average score, count the girls
4  for stu in score:
5      # if we find a boy
6      if stu[1] == "M":
7          continue   # this skips the boy
8      avg_F += stu[2]
9      count_F += 1
10 print("girl's average:", avg_F/count_F)
```

break

```
1  s = 'H2O'
2  digit_index = -1 # this will be -1 until we find a digit
3  for i in range(len(s)):
4      # if we find a digit
5      if s[i].isdigit():
6          digit_index = i
7          break   # this exits the loop
8  print("the index of digit:", digit_index)
```

Example – Bacteria Using `for` Loop

```
1  t = 0 # minutes
2  pop = 1000 # bacteria to start with
3  r = 0.21 # 21% growth per minute
4  while pop < 2000:
5      pop += pop * r
6      print(pop)
7      t += 1
8  print("Took", t, "minutes for the bacteria to double")
9  print("And the final population is", pop)
```

- Suppose that we calculate the growth of a bacterial colony with an equation: $P(t+1) = P(t) + rP(t)$
- where $P(t)$ is [while loop] t, and r is the growth rate
- We want to know how long it takes the bacteria to double their numbers

```
1  pop = 1000 # bacteria to start with
2  r = 0.21 # 21% growth per minute
3  for t in range(1,100):
4      pop += pop * r
5      print(pop)
6      if pop >= 2000:
7          break
8  print("Took", t, "minutes for the bacteria to double")
9  print("And the final population is", pop)
```

for loop

Example: Check Prime Number

```python
n = int(input("Please enter an integer (>1): "))
isPrime = True
i = 2
while i < n:
    if n % i == 0:
        isPrime = False
        break
    i = i + 1

if isPrime:
    print("%d is a Prime number." % (n))
else:
    print("%d is a not Prime number." % (n))
```

while loop

```python
n = int(input("Please enter an integer (>1): "))
isPrime = True
for i in range(2,n):
    if n % i == 0:
        isPrime = False
        break
if isPrime:
    print("%d is a Prime number." % (n))
else:
    print("%d is not a Prime number." % (n))
```

for loop

Best Friends: Strings and Lists

```
text = "I love Python..^^, and you?"
words = text.split()
print(words)            # ['I', 'love', 'Python..^^,', 'and', 'you?']
print(len(words))       # 5
print(words[2])         # Python..^^,
punc = [',', '?', '.', ' ', '^']
for i in range(len(words)):
    for c in punc:
        words[i] = words[i].replace(c, "")
print(words)            # ['I', 'love', 'Python', 'and', 'you']
words2 = text.split(",")
print(words2)           # ['I love Python..^^', ' and you?']

line = "a lot        of spaces"
separated = line.split()
print(separated)        # ['a', 'lot', 'of', 'spaces']
```

split() breaks a string into parts and produces a **list** of strings, which are treated as words.
We can **access** a particular word or **loop** through all the words

When you do not specify a **delimiter**, multiple spaces will be treated as *one* delimiter

Generating Random Numbers

- Python has a **random** module for drawing random numbers: first, you need to `import random`
- `random.random()` – Generate random numbers that are uniformly distributed in the interval [0,1)
- `random.uniform(a,b)` – Generate random numbers uniformly distributed in [a,b)

```
import random
print(random.random())          # 0.8102300732827268
print(random.random())          # 0.7179821822067495
print(random.random())          # 0.14725426561760568
print(random.uniform(-1,1))     # -0.0517625153803325
print(random.uniform(-1,1))     # 0.7905524017458676
print(random.uniform(-1,1))     # -0.12485136833326016
```

Generating Random Numbers

- `random.randint(a,b)` – Draw an integer from `[a,b]` (not a real number) (a and b are included)
- Different ways to randomly pick an element from a list
  - `randon.choice(list)` 擇一, `randon.shuffle(list)` 洗牌

```
import random
print(random.randint(1,10))     # 9
print(random.randint(-1,1))     # 1
print(random.randint(0,100))    # 22
suits = ["Club", "Diamond", "Heart", "Spade"]
print(random.choice(suits))     # Club
index = random.randint(0, len(suits)-1)
print(suits[index])             # Spade
random.shuffle(suits)   # shuffle the list randomly
print(suits[0])                 # Heart
```

Generating Random Numbers: Example

- Make a deck of cards, and draw three cards

```
1  import random
2  ranks = ['A', '2', '3', '4', '5', '6', '7',
3           '8', '9', '10', 'J', 'Q', 'K']
4  suits = ['C', 'D', "H", 'S']
5  # make a deck of cards
6  deck = []
7  for s in suits:
8      for r in ranks:
9          deck.append(s+r)
10 random.shuffle(deck)
11
12 # draw three cards at random
13 for i in range(3):
14     card = deck.pop(0)
15     print(card, " #cards=", len(deck))
```

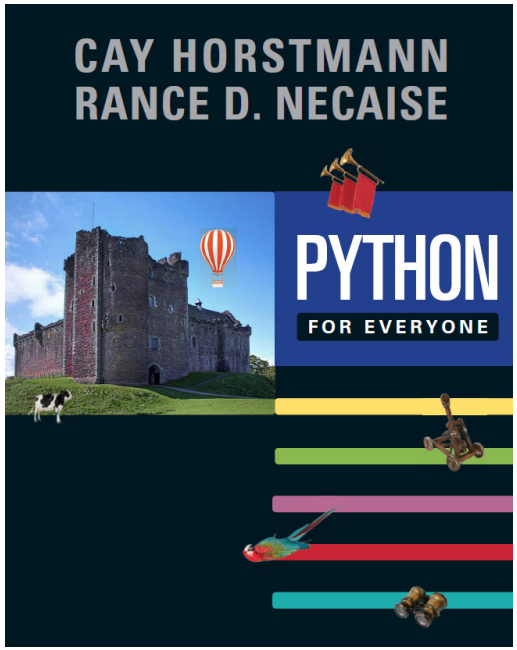
Output:

```
CA #cards= 51
H8 #cards= 50
SJ #cards= 49
```

Summary

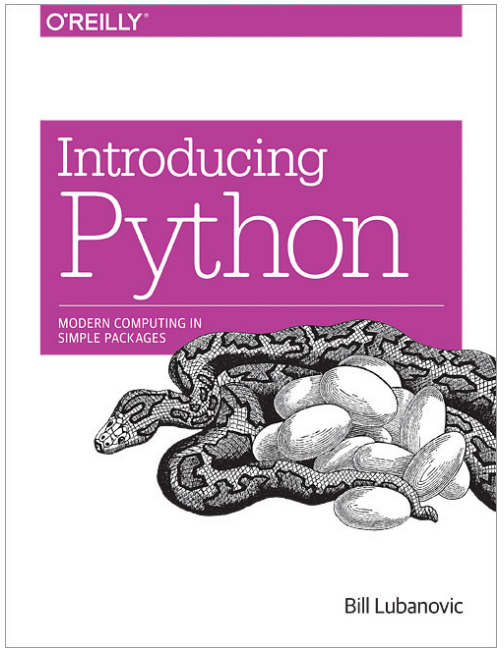
- for loop
- break and continue statements
- Nested for loop
- for list vs. string
- Generating random numbers

Suggested Reading



P.177 – P.200

P.277 – P.296

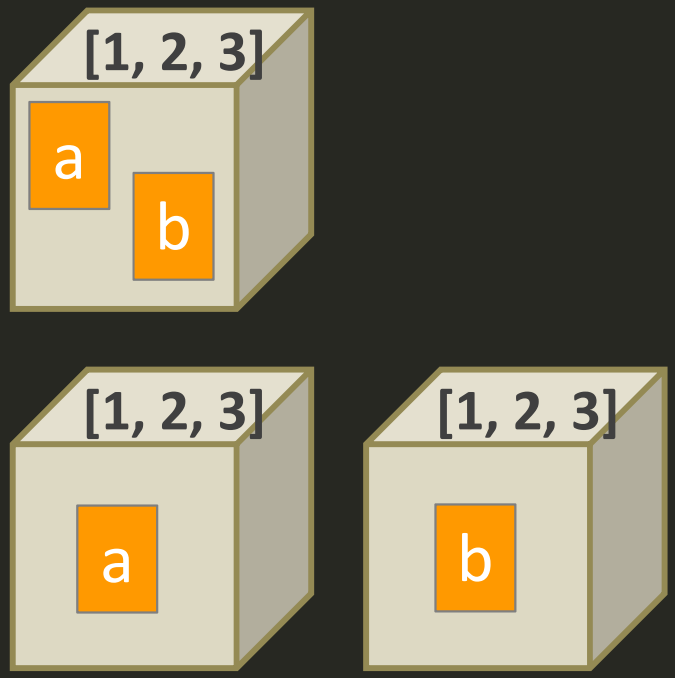


P.77 – P.79

`is` vs. `==`

- `x` **`is`** `y` returns `True` (reference equality)
  - If two references `x` and `y` point to the same object
- `x` **`==`** `y` returns `True` (value equality)
  - If the objects referred to by `x` and `y` have the same value

```
a = [1, 2, 3]
b = a
b is a     # True
b == a     # True
b = a[:]
b is a     # False
b == a     # True
```




```
a = 500
b = 500
a == b   # True
a is b   # True

a = []
b = []
a == b   # True
a is b   # False

a = None
b = None
a == b   # True
a is b   # True
```

`is` vs. `==`

- `id(name)`: see the object's (starting) address in memory

```
values = [67, 29, 35, 95]
prices = values                 # list aliasing
print(id(values), id(prices))   # 17701424 17701424
print(values == prices)         # True
print(values is prices)         # True

values = [67, 29, 35, 95]
prices = values[:]              # list copying
print(id(values), id(prices))   # 17701304 17701904
print(values == prices)         # True
print(values is prices)         # False

values = [67, 29, 35, 95]
prices = list(values)           # list copying
print(id(values), id(prices))   # 17701424 17719424
print(values == prices)         # True
print(values is prices)         # False
```

The Range Function

- The built-in function `range()` generates a **list** of integers
  - Return an **iterable object**
  - `list()` is used to force the iterable object to appear as a list
- It takes one, two, or three parameters:
  - `range(n)` generates a list of the integers in $[0, n)$
    - equivalent to: range(0, n)
  - `range(m,n)` generates a list of the integers in $[m, n)$
    - equivalent to: range(m, n, 1)
  - `range(m,n,s)` generates a list of the integers in $[m, n)$
- $s \neq 0$ is the step size
  - If $s > 0$, we must give $m < n$, else the list is empty
  - If $s < 0$, we must give $m > n$, else the list is empty